

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN



GRADO EN INGENIERÍA Y SISTEMAS DE DATOS

TRABAJO FIN DE GRADO

APLICACIÓN DE TÉCNICAS DE MACHINE LEARNING PARA LA
IDENTIFICACIÓN Y OPTIMIZACIÓN DE PATRONES EN 'SWING CHART' EN
MERCADOS FINANCIEROS

Lijun Zhou

2026

Grado en Ingeniería y Sistemas de Datos

TRABAJO DE FIN DE GRADO

Título: Aplicación de técnicas de Machine Learning para la identificación y optimización de patrones en 'Swing Chart' en mercados financieros

Autor: D. Lijun Zhou

Supervisión técnica:

Supervisión académica: Dr. Eduardo López Gonzalo

Departamento: Departamento de Señales, Sistemas y Radiocomunicaciones.

MIEMBROS DEL TRIBUNAL

Presidente:

Vocal:

Secretario:

Suplente:

Los miembros del tribunal arriba nombrados acuerdan otorgar la calificación de:

.....

Fecha de lectura:

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN



GRADO EN INGENIERÍA Y SISTEMAS DE DATOS

TRABAJO FIN DE GRADO

APLICACIÓN DE TÉCNICAS DE MACHINE LEARNING PARA LA
IDENTIFICACIÓN Y OPTIMIZACIÓN DE PATRONES EN 'SWING CHART' EN
MERCADOS FINANCIEROS

Lijun Zhou

2026

Resumen

El presente proyecto tiene como objetivo el desarrollo e implementación de un sistema basado en técnicas de Machine Learning orientado al análisis y detección de patrones en “Swing Charts”, una herramienta ampliamente utilizada en el análisis técnico de los mercados financieros. A partir de datos históricos y en tiempo real, el sistema generará representaciones de “Swing Charts” a partir de “Bar Charts” y aplicará algoritmos de aprendizaje supervisado y no supervisado para identificar configuraciones recurrentes, como dobles máximos, dobles mínimos o rupturas de tendencia.

El desarrollo se llevará a cabo principalmente en Python, utilizando librerías como pandas, scikit-learn y matplotlib, además de entornos de conexión con plataformas de trading como Interactive Brokers API (IBKR). El proyecto busca evaluar la capacidad predictiva de los modelos entrenados y su utilidad en la generación de señales de compra o venta dentro de una estrategia de trading algorítmico. Con ello, se pretende aportar un enfoque innovador que combine la solidez del análisis técnico clásico con la adaptabilidad de las técnicas modernas de inteligencia artificial.

Palabras clave

Trading algorítmico, predicción de mercados financieros, automatización, análisis en tiempo real, backtesting.

Summary

XXXXXXXXXX

Keywords

XXXXXXXXXX

Índice

1	Introducción y objetivos	1
1.1	Introducción	1
1.2	Objetivos	1
2	Estado del arte	4
2.1	XXXXXXXXXX	4
3	Desarrollo	5
3.1	Desarrollo del autómata	5
3.1.1	Carga y limpieza	6
3.1.2	Cálculo de características	6
3.1.3	Selección de características	7
3.1.4	Aprendizaje supervisado	7
3.1.5	Aprendizaje no supervisado	8
3.1.6	Análisis de resultados de entrenamiento y Backtest de las predicciones .	9
3.1.7	Gestión de cartera	9
3.1.8	Análisis de resultados de optimización y Backtest de las optimizaciones .	10
3.1.9	Simulación de Monte Carlo	10

3.1.10	Manejo del riesgo	10
3.1.11	Operación	11
3.1.12	Logging	12
3.2	Despliegue usando contenedores de Docker	12
3.2.1	Configuración del contenedor principal	12
3.2.2	Bases de datos	12
3.2.3	Visualización	13
3.2.4	Alertas y avisos	13
3.2.5	Orquestación y MLOps	14
4	Resultados	15
4.1	Resultados	15
5	Conclusiones y líneas futuras	16
5.1	Conclusiones	16
5.2	Líneas futuras	16
6	Aspectos éticos, económicos, sociales y ambientales	17
6.1	Introducción	17
6.2	Descripción de impactos relevantes relacionados con el proyecto	17
6.3	Análisis detallado de alguno de los principales impactos	17
6.4	Conclusiones	17
7	Presupuesto económico	18
	Bibliografía	20

Anexos	21
A XXXXXXXXXXXXXXXXXXXX	21

Listado de figuras

3.1	Añadir img con algunos plots de las características más usadas como modelos solos	6
3.2	Añadir img con una simulación del Backtest	9
3.3	Añadir img con una simulación de Monte Carlo	10
3.4	Añadir img dashboard grafana	13
3.5	Añadir img Alertmanager o Slack	14
4.1	Añadir img con equity curve, max drawdown y sharpe ratio	15

Listado de tablas

1.1	Características principales de los contratos de futuros analizados	1
3.1	Tabla comparativa de parámetros de los modelos	7
3.2	Tabla comparativa de gestión de riesgo	10

Todo list

☐	Mejorar el texto	1
☐	Justificar cambio de ML tradicional a DL	2
☐	Revisar clustering para comparar futuros y para segmentación de modelos	3
☐	Dejar UL para futuro mejor	3
☐	Explicar mejor cómo funciona la función de inferencia	6
☐	Probar a ver otros métodos de votación además de usar la media	7
☐	Pasar a planes de futuro	8
☐	Revisar esta afirmación	8
☐	Explicar qué es Pyomo	9
☐	Explicar qué es Sharpe Ratio	10
☐	Implementar la gestión de cartera	10
☐	Comparar respecto a usar todo el capital en un solo activo	10
☐	Ver donde colocar mejor el último párrafo	11
☐	Ejemplos de donde añadir logging	12
☐	Ver si con sqlite vale o pasar directamente a MySQL	12
☐	Tanto Airflow como MLflow, ver si hay tiempo para implementarlos	14
☐	No me da la RAM, hay que separar, despliegue de solo MLflow y Airflow	14

- ☐ Añadir grafica de la evolución del equity, comparativa contra Robotrader 16
- ☐ Añadir Monte Carlo 16
- ☐ Preguntar si demasiadas lineas futuras demuestra poco o mucho interes en el trabajo 16

Glosario

OHLCV - Precios Open High Low y Close y Volumen

XXXX - XXXXXXXXX

Introducción y objetivos

1.1 Introducción

El trading tradicional tiene el inconveniente de depender de los sentimientos humanos de cada persona.

Por ello se creó el trading algorítmico donde las decisiones se toman por funciones matemáticas y umbrales fijados.

El siguiente paso dentro del trading algorítmico es el uso de modelos de aprendizaje automático que se entrenan usando diferentes características derivadas de la información obtenida de los diferentes activos financieros

Para empezar este trabajo tomé como referencia el libro [1]

Mejorar el texto

1.2 Objetivos

Tabla 1.1: Características principales de los contratos de futuros analizados

Símbolo	Nombre	Mult.	Tick Size	Tick Value	Margen Init
AD	Australian Dollar	100000	0.00005	\$5	\$2503.59
BP	British Pound	62500	0.0001	\$6.25	\$2585.50
CL	Crude Oil WTI	1000	0.01	\$10	\$28336.53
EC	Euro FX	125000	0.00005	\$6.25	\$3671.13
ES	E-mini S&P 500	50	0.25	\$12.5	\$32380.88
GC	Gold	100	0.1	\$10	\$44136.61
MFXI	Micro Euro FX (M6E)	12500	0.0001	\$1.25	\$367.60
NG	Natural Gas	10000	0.001	\$10	\$6592.86
NQ	E-mini Nasdaq 100	20	0.25	\$5	\$60368.33
YM	E-mini Dow Jones	5	1	\$5	\$17296.56
ZB	30Y US Treasury Bond	1000	0.03125	\$31.25	\$4259.66
ZN	10Y US Treasury Note	1000	0.015625	\$15.625	\$2156.85

ZS	Soybean Futures	5000	0.0025	\$12.5	\$4077.58
----	-----------------	------	--------	--------	-----------

El objetivo de este presente trabajo es lograr desarrollar un sistema que opere de manera autónoma una cartera con un valor de un millón de dolares para obtener el mayor beneficio posible asumiendo el mínimo riesgo. Como se puede observar en la tabla anterior el sistema operará sobre 13 futuros distintos que incluyen divisas, materias primas, índices bursátiles y bonos del Tesoro estadounidense, proporcionando así una exposición diversificada a diferentes clases de activos y comportamientos de mercado.

Para lograr este objetivo he decidido desarrollar una arquitectura basada en modelos de aprendizaje automático que ayudan a decidir las acciones a realizar, se emplea tanto modelos de aprendizaje supervisado como no supervisado, cada uno de ellos se detallará en los siguientes capítulos correspondientes. Como resumen de la arquitectura se usarán modelos de aprendizaje profundo como modelos de aprendizaje supervisado y modelos tradicionales

Justificar cambio de ML tradicional a DL

El núcleo del sistema funciona de la siguiente manera

1. Se cargan los datos minuto a minuto de uno de los futuros
2. Se estandariza el tiempo que aparece en las muestras a UTC
3. Se eliminan los datos que son claramente erróneos (fines de semana)
4. Se recortan los dataset para tener los mismos periodos para todos los futuros
5. Se añade el día de negociación al que pertenece cada dato
- Visualizar los datos originales limpios
6. Obtener la información actualizada de los diferentes futuros
7. Muestreo de los datos en diferentes periodos
8. Cálculo de los datos acumulados a partir de los datos originales
9. Cálculo de las características que se usarán para entrenar los modelos de aprendizaje automático
10. Analizar las nuevas características para revisar si la información tiene sentido
11. Decidir algunas de las características que no se usarán manualmente por criterios como la no estacionalidad
12. Calcular la variable objetivo que los modelos supervisados tratarán de predecir

13. Filtrar más características usando modelos supervisados tradicionales que permitan mayor explicabilidad
14. Calcular los parámetros que se utilizaran para definir el entrenamiento de los modelos
15. Entrenar los modelos por separado
16. Durante cada época del entrenamiento se usan los modelos no supervisado para detectar el régimen de mercado actual y usar esta información como características adicionales para el modelo
17. Búsqueda de los umbrales de confianza más adecuados para cada modelo y evaluación de los resultados de la predicción usando métricas estándar de clasificación y métricas dedicadas de finanzas
18. Guardar los mejores modelos

Revisar clustering para comparar futuros y para segmentación de modelos

Dejar UL para futuro mejor

Estado del arte

2.1 XXXXXXXXXXXX

Desarrollo

3.1 Desarrollo del autómata

El desarrollo del sistema se divide en diferentes fases dentro de la ETL, destacando la carga de datos históricos, su limpieza y filtrado, su transformación en las diferentes características para alimentar a los modelos de aprendizaje automático.

El objetivo es poder generalizar el código, que no importe de qué activos sean los datos el flujo de datos puede reutilizarse.

Para lograr este objetivo he decidido implementar la lógica del código dentro de un paquete instalable de python usando `pip install -e` y he decidido usar la siguiente estructura organizativa.

```

/
├── paths.py
├── get_info
├── data
│   ├── loader.py
│   ├── cleaner.py
│   ├── resampler.py
│   └── to_parquet.py
├── features
├── supervised
├── unsupervised
├── portfolio
├── risk
├── execution
├── logging
└── retrain
    
```

Hay funciones que están en algunos de los directorios que pueden servir para varias etapas del proceso, principalmente aquellas encargadas de obtener los datos desde los archivos.

A continuación paso a explicar cada una de las fases del flujo de los datos y las funciones utilizadas en ellas.

3.1.1 Carga y limpieza

Se me han facilitado los datos históricos OHLCV de distintos futuros con los que voy a operar.

En un primer análisis me doy cuenta de que los datos temporales no están regularizados al formato UTC, por lo que es el primer cambio que realizo. Usando de referencia las horas de apertura y cierre de cada futuro, agrupo el volumen por horas para cada activo puedo detectar las horas con poca o nula actividad, con ello puedo inferir la zona horaria usada.

Explicar mejor cómo funciona la función de inferencia

Después, decido filtrar los datos que claramente no deberían de estar, estos son los que aparecen en los fines de semana. Los pedidos realoizados en las horas de descanso las mantengo por que pueden haber sido retrasos en la ejecución por lo que son operaciones reales no realizadas en el momento indicado.

Una vez estandarizado el tiempo aprovecho la función de inferencia y añado una primera columna derivada que indica el día de *trading* aunque al ser una fecha no se usará directamente para el entrenamiento si me ayudará para calcular valores acumulados diarios.

Por otra parte como mi intención es obtener métricas de comparación entre activos decido recortar el número de filas disponibles en los diferentes archivos para tener un conjunto homogéneo. Además para mejorar el rendimiento y disminuir el tamaño de los archivos convierto los datos que estaban en formato csv a parquet.

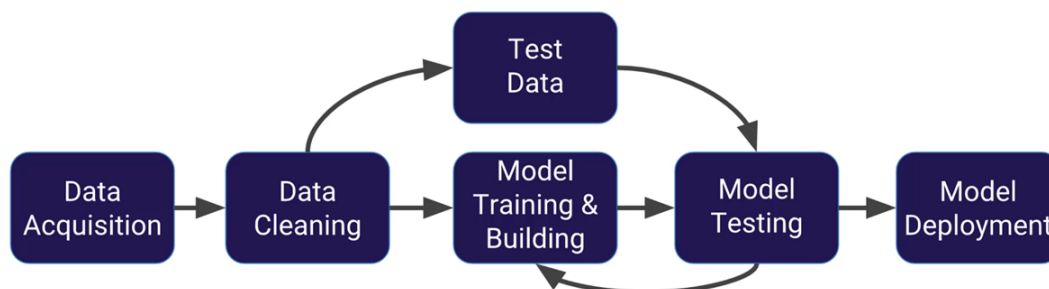


Figura 3.1: Añadir img con algunos plots de las características más usadas como modelos solos

3.1.2 Cálculo de características

Una vez con los datos OHLCV limpios, llega el momento de aplicar diferentes funciones de diversas librerías para obtener características derivadas que servirán para alimentar a los mo-

delos.

Hay características de diferentes tipos, momento, estructural, temporal, de tendencia, de volatilidad, de volumen y características derivadas de otras. Al ser una estrategia multidía también he hecho uso de los swing charts y de características derivadas de ellas, además de otras características obtenidas de comparar diferentes activos

Para obtener estas características primero agrupé los datos que venían minuto a minuto a agrupaciones de 4 horas, de esto modo se evita tener el ruido de señales tan cortas y es un buen número para una estrategia interdía donde una posición puede estar abierta varios días

3.1.3 Selección de características

Una vez calculadas todas las características es necesario reducir ese número porque es posible que muchas de ellas estén interrelacionadas

Para ello se aplican varias técnicas, empezando por la más básica, un filtro de alta correlación, además de eliminar las características que puedan predecir el futuro, las no cíclicas o no estacionarias pueden confundir a los modelos indicando que en valor que quizás representaba el día de la semana sea considerado como más importante a mayor número

También filtro usando el algoritmo de LASSO usando un modelo de aprendizaje automático clásico de sklearn que permití mayor explicabilidad

3.1.4 Aprendizaje supervisado

El uso de modelos de aprendizaje supervisado se centra en modelos de clasificación binaria, en este caso en dos etapas, y dentro de cada etapa se utilizan dos modelos de aprendizaje profundo usando PyTorch, LSTM y GRU. Cada uno de los modelos utilizados tiene una función, LSTM ..., por su parte GRU. Para ambas fases se realiza una votación equitativa, si la media supera el umbral indicado para cada fase se pasa a la siguiente

Probar a ver otros métodos de votación además de usar la media

Existe una tercera etapa, en este caso de regresión que sigue la misma lógica, usando los dos modelos que servirá para ayudar

Tabla 3.1: Tabla comparativa de parámetros de los modelos

1	2	3	4	5
---	---	---	---	---

3.1.5 Aprendizaje no supervisado

Pasar a planes de futuro

Además de usar modelos de aprendizaje supervisado tenía la idea de usar modelos no supervisados para un par de tareas

En primer lugar, para agrupar los diferentes activos en diferentes grupos. En vez de realizar agrupaciones arbitrarias, por ejemplo, todas las monedas juntas, dejo que el sistema decida cuales son los activos más parecidos. De este modo podría generar más características derivadas de la comparación de los activos, basandome en el concepto de *Pairs Trading* donde se comparan un par de activos que se saben que son correlados y cointegrados, y si uno de ellos sube más que el otro se abre una posición en corto y si uno de ellos baja más que el otro se abre una posición en largo.

El problema que le encuentro a este primer caso de uso es la disparidad de datos, por una parte los datos históricos proporcionados no son de las mismas fechas, esto se soluciona fácilmente al recortar los datos más antiguos ya que además podrían causar problemas a los modelos. Pero donde existen más problemas es el tema de los horarios de las diferentes bolsas, la idea sería usar la función de rellenado hacia adelante para los momentos donde faltasen datos de alguno de los futuros. Otra opción sería comparar cada futuro con el centroide (el centro de cada cluster) del cluster al que pertenece cada futuro y el resto de centroides, de este modo se puede evitar problemas por la disparidad del número de futuros en cada cluster.

Además de este problema está la necesidad de tomar la decisión de qué tipo de modelo no supervisado usar, si uno donde se predefine el número de clusters o uno donde el propio modelo decide el número (Decidir entre modelos paramétricos (k fijo) o modelos no paramétricos (k variable))

El segundo uso es el de segmentar los regímenes de mercado. Con esta información hay dos opciones de acción, por una parte, usarlo como una característica más para complementar al resto, aunque puede no ser tan útil al usar modelos de aprendizaje supervisado profundo que al estar diseñados para manejar series temporales pueden inferir los regímenes de mercado por si solos. La segunda opción es usar la diferencia de regímenes para entrenar modelos dedicados para cada régimen consiguiendo modelos más especializados en cada régimen. Este uso tiene los mismos problemas que el primero

Revisar esta afirmación

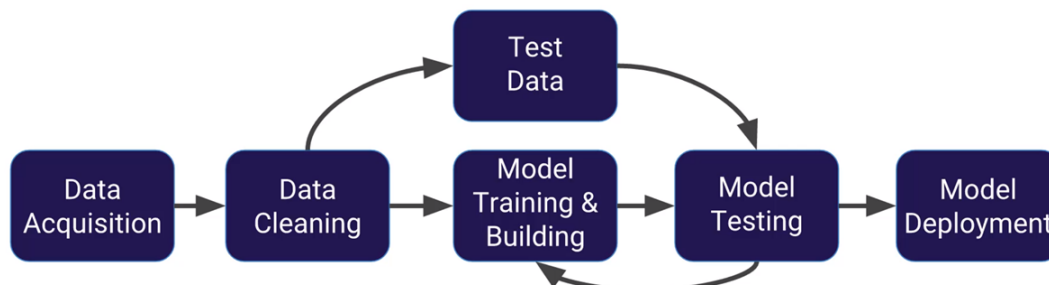


Figura 3.2: Añadir img con una simulación del Backtest

3.1.6 Análisis de resultados de entrenamiento y Backtest de las predicciones

3.1.7 Gestión de cartera

Una vez entrenados los modelos para cada futuro es necesario coordinarlos todos para que no hayan problemas de liquidez. Para ello en un primer momento decidí usar un método más rudimentario y manual para después pasar a la optimización de la cartera usando Pyomo

En el primer caso se realizan los siguientes pasos

1. Realizar todas las predicciones para todos los futuros disponibles en el momento temporal
2. Ordenar por nivel de confianza total
3. Iterar sobre cada predicción por este orden e ir comprobando si cumple con los requisitos para poder operar como la disponibilidad de capital suficiente
4. Ejecutar las diferentes operaciones

Una vez que tuve el sistema funcionando paso a implementar un optimizador usando Pyomo

Explicar qué es Pyomo

Para ello sigo los siguiente paso para una optimización de un objetivo, en este caso maximizar el *Sharpe Ratio*

1. Definir variables

2. ...

Explicar qué es Sharpe Ratio

Implementar la gestión de cartera

Tabla 3.2: Tabla comparativa de gestión de riesgo

1	2	3	4	5
---	---	---	---	---

3.1.8 Análisis de resultados de optimización y Backtest de las optimizaciones

Comparar respecto a usar todo el capital en un solo activo

3.1.9 Simulación de Monte Carlo

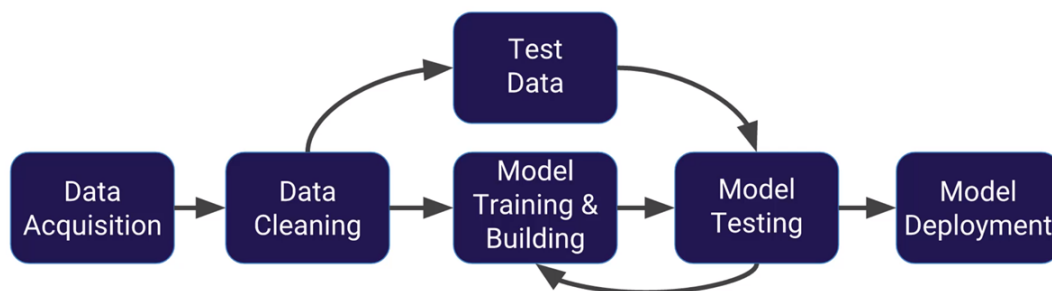


Figura 3.3: Añadir img con una simulación de Monte Carlo

3.1.10 Manejo del riesgo

Una vez entrenados los modelos y antes de pasar a la operación es necesario definir el riesgo que uno está dispuesto a tener. La métrica más importante en este sentido es el *Max Drawdown*, que define la mayor caída del valor neto de la cartera. En mi caso lo defino como un valor porcentual. Relacionado con esto está el porcentaje máximo de activo en posiciones abiertas

que estoy dispuesto a mantener simultáneamente, es decir, el porcentaje máximo del equity expuesto. Esta métrica define el límite superior de capital que puede estar comprometido en operaciones abiertas en un momento dado, independientemente del apalancamiento implícito de cada instrumento *Exposure Limit*.

Por otra parte a la hora de operar hay que tener en cuenta que el spread al inicio y al final de las sesiones puede ser mayor que durante el resto del tiempo, por ello al tener información de las horas de mercado es posible decidir que para esos instantes temporales en vez de operar a en punto atrasar la toma de decisiones a 15 minutos después o una vez que se supere un umbral de volumen.

Por último como medida para estar al día con los datos de los futuros, cada semana se entrenan nuevos modelos o se afinan los modelos actuales y se comparan con los modelos actuales para comprobar cuales son mejores predictores de los datos de la última semana o mes

Ver donde colocar mejor el último párrafo

3.1.11 Operación

Una vez con todo preparado llega el momento de unir todo en un único código que será el que se conecte a la API de Interactive Brokers para realizar las operaciones.

La operación tendrá los siguientes pasos

1. Conexión a la API de Interactive Brokers
2. Lectura de los futuros sobre los que se van a operar
3. Descarga de los datos necesarios para realizar las predicciones
4. Carga de la información de configuración del sistema
5. Carga de los modelos, los escaladores, y la configuración individual de cada futuro
6. Cada X minutos realizar las predicciones de cada futuro

Como los futuros tienen fecha de caducidad obtengo una lista para cada futuro con el número de contratos activos y los ordeno para obtener el contrato con mayor volumen, minimizando el spread. En el caso de estar corriendo el código durante varios días seguidos el sistema realiza esta consulta una vez al día. En caso de que sigan habiendo contratos abiertos existen dos posibilidad dependiendo de cómo esté configurado, en el primer caso cierra esos contratos directamente, en el segundo caso comprueba la diferencia de volumen y si es menor de un umbral decide seguir con el contrato anterior, si no, hace como en el primer caso y cierra las posiciones.

3.1.12 Logging

No es una fase per se, es algo que se debe implementar en las diferentes etapas del flujo para saber qué está pasando en cada parte y para poder llevar un registro de todo.

Ejemplos de donde añadir logging

3.2 Despliegue usando contenedores de Docker

Una vez que se ha desarrollado el autómata y se ha comprobado que su funcionamiento es correcto, he decidido añadir el proyecto a un contenedor de docker para facilitar el despliegue en otros equipos

3.2.1 Configuración del contenedor principal

Por cómo he desarrollado el sistema y por velocidad todo va a estar incluido en un único contenedor.

No requiere demasiados cambios ya que voy a copiar los códigos desarrollados dentro del contenedor.

De este modo se logra tener un entorno estático donde se han seleccionado todas las versiones de los diferentes paquetes usados en el proyecto

Además, esto permite agrupar todas las funcionalidades que quiero implementar en un mismo lugar, diferentes contenedores con funciones dedicadas.

3.2.2 Bases de datos

Al tener el proyecto dentro de un contenedor veo lógico tener una base de datos con diferentes tablas para guardar diferentes métricas. Por facilidad voy a guardar las estadísticas en archivos sqlite, aunque a futuro existe la posibilidad de pasar a MySQL si necesito más rendimiento

Porñ

Ver si con sqlite vale o pasar directamente a MySQL

3.2.3 Visualización

Por último una vez que todo funciona en contenedores he decidido añadir un sistema de visualización de toda la información que se recopila en directo y en diferido

Para ello uso varios contenedores, pushgateway, prometheus, loki, promtail, y grafana.

Prometheus y Pushgateway gestionan métricas; Promtail y Loki gestionan logs. Pushgateway y Promtail son los recolectores, mientras que Prometheus y Loki son los motores de almacenamiento y consulta.

Toda la información se consume después en un *dashboard* en grafana donde se agrupa toda la información.

En la visualización se observan diferentes métricas empezando por las relacionadas con el rendimiento del sistema. Equity curve, max drawdown, sharpe ratio, volatilidad diaria, profit factor, win rate, rendimiento por futuro. Por otra parte están

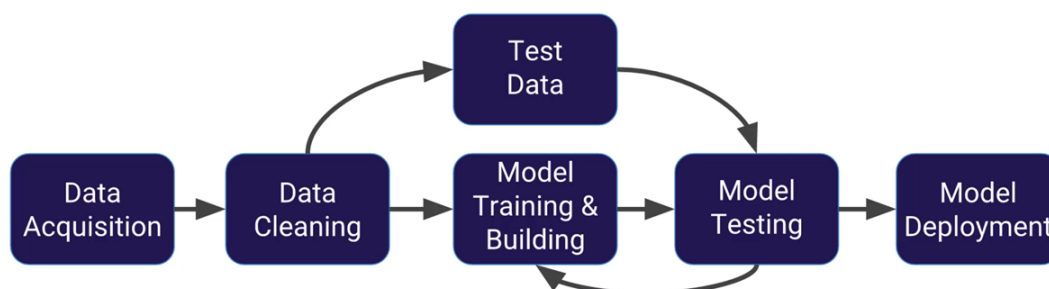


Figura 3.4: Añadir img dashboard grafana

3.2.4 Alertas y avisos

Por último una vez que todo funciona en contenedores he decidido añadir un sistema de visualización de toda la información que se recopila en directo y en diferido

Uso Prometheus, Grafana y Alertmanager este último conectado a otro servicio que se conecta a la API de whatsapp

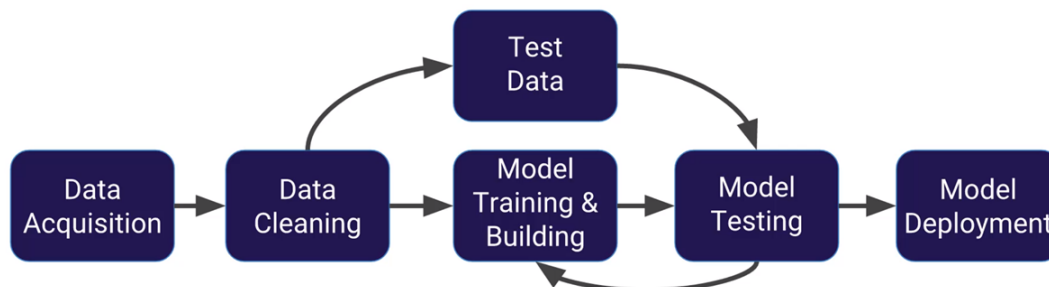


Figura 3.5: Añadir img Alertmanager o Slack

3.2.5 Orquestación y MLOps

Tanto Airflow como MLflow, ver si hay tiempo para implementarlos

Una vez desplegado un sistema robusto que cuenta con un autómata funcional y con alertas tanto de seguridad como para mostrar información, decido implementar el uso de Airflow como orquestador.

En general el uso de Airflow está muy ligado al uso de MLflow ya que será el uso principal de esta herramienta por mi parte.

En MLflow

No me da la RAM, hay que separar, despliegue de solo MLflow y Airflow

Resultados

4.1 Resultados

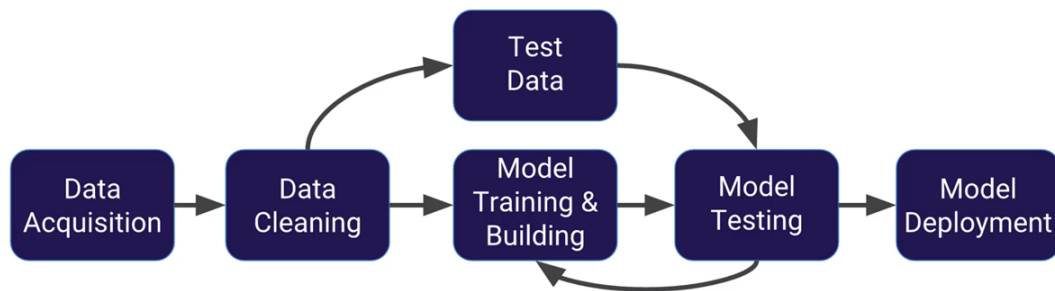


Figura 4.1: Añadir img con equity curve, max drawdown y sharpe ratio

Conclusiones y líneas futuras

5.1 Conclusiones

Añadir grafica de la evolución del equity, comparativa contra Robotrader

Añadir Monte Carlo

5.2 Líneas futuras

Preguntar si demasiadas lineas futuras demuestra poco o mucho interes en el trabajo

Por limitaciones del tiempo dejo pendiente dos mejoras

Por una parte añadir una interfaz gráfica independiente de grafana para tener personalización total

Y por otra parte añadir en esa misma página un chatbot usando RASA o un LLM para responder preguntas a partir de toda la información disponible

Aspectos éticos, económicos, sociales y ambientales

6.1 Introducción

6.2 Descripción de impactos relevantes relacionados con el proyecto

6.3 Análisis detallado de alguno de los principales impactos

6.4 Conclusiones

Presupuesto económico

COSTE DE MANO DE OBRA (coste directo)		Horas	Precio/hora	Total
		360	15 €	5.400 €
COSTE DE RECURSOS MATERIALES (coste directo)	Precio de compra	Uso en meses	Amortización (en años)	Total
Ordenador personal (Software incluido)	1.500,00 €	6	5	150,00 €
Impresora láser	500,00 €	6	5	50,00 €
Otro equipamiento		X	X	X €
COSTE TOTAL DE RECURSOS MATERIALES				200,00 €
GASTOS GENERALES (costes indirectos)	15 %	sobre CD		840,00 €
BENEFICIO INDUSTRIAL	6 %	sobre CD+CI		336.00 €
MATERIAL FUNGIBLE				
Impresión				100,00 €
Encuadernación				300,00 €
SUBTOTAL PRESUPUESTO				7.176,00 €
IVA APLICABLE			21 %	1.507,00 €
TOTAL PRESUPUESTO				8.683,00 €

Bibliografía

- [1] I. Trullàs, *Trading algorítmico con Python: Desarrolla tus habilidades en el Trading Algorítmico. Con ejemplos e integración a MetaTrader5*. Amazon Kindle Direct Publishing, 2024. Kindle edition.

Anexos

A XXXXXXXXXXXXXXXXXXXX